

# 100 задач на map в Go

## 1.

Напишите функцию `CountInts(nums []int) map[int]int`, которая возвращает количество вхождений каждого числа.

## 2.

Напишите функцию `LastIndex(words []string) map[string]int`, которая для каждой строки хранит индекс ее последнего вхождения.

## 3.

Напишите функцию `Unique(nums []int) []int`, которая удаляет повторы, сохраняя порядок первых вхождений. Для проверки уникальности используйте `map[int]struct{}`.

## 4.

Напишите функцию `ContainsAll(have, need []string) bool`. Она должна вернуть `true`, если каждое значение из `need` встречается в `have` хотя бы один раз.

## 5.

Напишите функцию `SortedKeys(m map[string]int) []string`, которая возвращает ключи карты в лексикографическом порядке.

## 6.

Напишите функцию `SumValues(m map[string]int) int`, которая вычисляет сумму всех значений карты. Пустая и `nil`-карта должны давать ноль.

## 7.

Напишите функцию `Invert(m map[string]int) (map[int]string, error)`. Если два исходных ключа имеют одинаковое значение, функция должна вернуть ошибку.

## 8.

Напишите функцию `MergeCounts(a, b map[string]int) map[string]int`, которая создает новую карту и складывает значения совпадающих ключей. Исходные карты изменять нельзя.

## 9.

Напишите функцию `DeleteZeroValues(m map[string]int)`, которая на месте удаляет все элементы с нулевым значением. Функция должна корректно работать с `nil`.

## 10.

Напишите функцию `Clone(m map[string][]int) map[string][]int`, создающую полностью независимую копию карты и каждого вложенного слайса.

## 11.

Напишите функцию `Equal(a, b map[string]int) bool`, которая сравнивает две карты по наборам ключей и значениям, не используя `reflect.DeepEqual`.

**12.**

Напишите функцию `Difference(a, b []int) []int`, которая возвращает уникальные значения, присутствующие в `a`, но отсутствующие в `b`. Порядок результата должен совпадать с порядком первых вхождений в `a`.

**13.**

Напишите функцию `Intersection(a, b []string) []string`, которая возвращает уникальное пересечение двух слайсов в порядке первых вхождений в `a`.

**14.**

Напишите функцию `SymmetricDifference(a, b []int) []int`, которая возвращает значения, встречающиеся только в одном из двух наборов. Результат отсортируйте по возрастанию.

**15.**

Напишите функцию `UniqueFold(words []string) []string`, удаляющую дубликаты без учета регистра, но сохраняющую написание первого вхождения. Используйте Unicode-преобразование регистра.

**16.**

Напишите функцию `MostFrequent(nums []int) (value int, count int, ok bool)`. При равной частоте выбирайте меньшее число; для пустого входа возвращайте `ok == false`.

**17.**

Напишите функцию `FirstUniqueRune(s string) (rune, bool)`, которая находит первую руну, встретившуюся ровно один раз.

**18.**

Напишите функцию `AreAnagrams(a, b string) bool`. Игнорируйте регистр и все Unicode-пробельные символы, остальные руны учитывайте точно.

**19.**

Напишите функцию `WordFrequency(text string) map[string]int`. Словом считается непрерывная последовательность Unicode-букв или цифр; регистр не учитывается.

**20.**

Напишите функцию `TopKWords(text string, k int) []string`, возвращающую `k` самых частых слов. При равной частоте слово, меньшее лексикографически, должно идти раньше.

**21.**

Напишите функцию `GroupByParity(nums []int) map[string][]int` с ключами `even` и `odd`. Относительный порядок чисел в каждой группе сохраните.

**22.**

Напишите функцию `GroupByLength(words []string) map[int][]string`, группирующую строки по количеству рун, а не байт.

**23.**

Напишите функцию `GroupAnagrams(words []string) [][]string`. Порядок групп определяется первым словом каждой группы, порядок слов внутри группы должен совпадать с исходным.

**24.**

Напишите функцию `MapValues(m map[string]int, f func(int) int) map[string]int`, применяющую `f` к каждому значению и не изменяющую исходную mapу.

**25.**

Напишите функцию `FilterMap(m map[string]int, keep func(string, int) bool) map[string]int`, оставляющую только элементы, для которых `keep` вернула `true`.

**26.**

Напишите функцию `Compose(first map[string]string, second map[string]int) map[string]int`. В результат должен попасть ключ `k`, только если `first[k]` существует как ключ в `second`.

**27.**

Напишите функцию `InnerJoin(left map[int]string, right map[int]int) []Row`, где `Row` содержит `ID`, `Name`, `Score`. Верните строки только для общих `ID`, отсортированные по `ID`.

**28.**

Определите типы `ChangeKind`, `Change` и напишите `Diff(old, current map[string]string) []Change`. Результат должен отдельно описывать добавленные, удаленные и измененные ключи и быть детерминированным.

**29.**

Определите тип `PatchValue`, позволяющий различать операции установить строку, установить пустую строку и удалить ключ. Напишите `ApplyPatch(dst map[string]string, patch map[string]PatchValue)`.

**30.**

Реализуйте тип `BiMap` для взаимно однозначного соответствия `string <-> int` с методами `Put`, `GetByString`, `GetByInt`, `DeleteByString`. `Put` должен атомарно отклонять конфликт любого направления.

**31.**

Дан `User{ID int, Email string, Name string}`. Напишите `IndexUsers(users []User) (map[int]User, error)`, отклоняющую повторяющиеся `ID`.

**32.**

Даны `Order{ID, UserID int; Amount int64}`. Напишите `OrdersByUser(orders []Order) map[int][]Order`. Порядок заказов внутри групп сохраните.

**33.**

Даны заказы с полями `UserID`, `Amount` и `Status`. Напишите функцию, возвращающую по каждому пользователю структуру с количеством оплаченных заказов, общей суммой и максимальной суммой.

**34.**

Дан слайс пользователей с полями `ID` и `City`. Постройте индекс `map[string]map[int]struct{}`. Реализуйте добавление, удаление и перенос пользователя между городами без пустых внутренних множеств.

**35.**

Реализуйте каталог товаров с основным хранилищем `map[int]Product` и индексами по категории и производителю. Методы `Add`, `Update`, `Delete` должны сохранять согласованность всех индексов при ошибках.

**36.**

В мапе хранится `map[int]User`, а `User` является структурой. Напишите `RenameUser(users map[int]User, id int, name string) error`, корректно изменяющую имя существующего пользователя.

**37.**

Напишите `Memoize(f func(int) int) func(int) int` для однопоточного кода. Обертка должна вызывать `f` не более одного раза для каждого аргумента, включая аргументы, для которых результат равен нулю.

**38.**

Дана структура `Record{Country, Number string}`. Найдите дубликаты по составному ключу из двух полей. Верните группы индексов только для ключей, встретившихся больше одного раза.

**39.**

Реализуйте разреженное двумерное поле `Grid[T]` на основе `map[Point]T`, где `Point` является ключом-структурой. Добавьте методы `Set`, `Get`, `Delete` и `Bounds`.

**40.**

Напишите функцию построения тепловой карты `map[Cell]int` по точкам с координатами `float64`. Размер ячейки задается параметром; отрицательные координаты должны распределяться математически корректно.

**41.**

Реализуйте множество `Set[T comparable]` на основе мапы. Нужны методы `Add`, `Remove`, `Contains`, `Len`, `Clone`, а также функции `Union`, `Intersection` и `Difference`.

**42.**

Даны роли, каждая содержит набор разрешений. Напишите функцию, вычисляющую итоговый набор разрешений пользователя по списку ролей, и функцию, возвращающую отсутствующие обязательные разрешения.

**43.**

Напишите `Flatten(m map[string]map[string]int) map[string]int`, соединяющую уровни ключом вида `outer.inner`. Если такое представление создает коллизию, верните ошибку.

**44.**

Напишите `DeepClone(m map[string]map[int][]byte) map[string]map[int][]byte`. Ни одна внутренняя мапа или слайс результата не должна разделять изменяемую память с оригиналом.

**45.**

Дана разреженная матрица `map[int]map[int]int`. Реализуйте сложение двух матриц, не сохраняя нулевые элементы и пустые строки.

**46.**

Для разреженной матрицы `map[int]map[int]int` реализуйте транспонирование. Результат должен быть независим от исходной матрицы.

**47.**

Реализуйте ориентированный граф как `map[string]map[string]struct{}`. Добавьте вершину, добавьте ребро, удалите ребро и удалите вершину вместе со всеми входящими ребрами.

**48.**

Напишите `BFSDistances(graph map[string][]string, start string) map[string]int`, возвращающую длину кратчайшего пути до каждой достижимой вершины.

**49.**

Напишите функцию поиска компонент связности неориентированного графа. Вершины и каждая компонента в результате должны быть отсортированы для детерминированности.

**50.**

Реализуйте топологическую сортировку ориентированного графа. Если граф содержит цикл, верните ошибку; если допустимо несколько результатов, выбирайте лексикографически минимальный.

**51.**

Напишите функцию кратчайшего пути в невзвешенном графе, возвращающую не только длину, но и последовательность вершин от `start` до `target`.

**52.**

Реализуйте алгоритм Дейкстры для графа `map[string]map[string]int64`. Отрицательный вес должен считаться ошибкой; защититесь от переполнения суммы расстояний.

**53.**

Дан набор авиабилетов `Ticket{From, To string}`; одинаковые билеты допустимы. Восстановите маршрут, использующий каждый билет ровно один раз. Среди возможных маршрутов верните лексикографически минимальный.

**54.**

Напишите функцию обнаружения цикла в ориентированном графе, которая возвращает конкретную последовательность вершин цикла, включая повтор стартовой вершины в конце.

**55.**

Реализуйте `DisjointSet[T comparable]` с операциями `MakeSet`, `Find`, `Union` и `Connected`. Используйте мапы, сжатие путей и объединение по размеру.

**56.**

Реализуйте Unicode-trie, у которого переходы хранятся в `map[rune]*Node`. Нужны операции вставки, точного поиска и удаления слова с очисткой ненужных узлов.

**57.**

Расширьте `trie` методом `Autocomplete(prefix string, limit int) []string`. Результат должен быть лексикографически отсортирован и не зависеть от порядка обхода мапы.

**58.**

Реализуйте `WordLadder(begin, end string, dictionary []string) []string`: за шаг можно заменить одну руну, все промежуточные слова должны быть в словаре. Для ускорения постройте карту шаблонов; верните один кратчайший путь.

**59.**

Даны проводки `Entry{Account string; Delta int64}`. Напишите функцию расчета балансов с проверкой переполнения `int64` и отдельным списком счетов, ушедших в минус.

**60.**

Реализуйте обработчик денежных переводов с картой балансов и множеством обработанных `RequestID`. Повтор запроса не должен менять состояние; нехватка денег и переполнение должны оставлять все карты без изменений.

**61.**

Реализуйте однопоточную in-memory транзакцию над `map[string]string` с операциями `Begin`, `Get`, `Set`, `Delete`, `Commit`, `Rollback`. Вложенные транзакции запрещены.

**62.**

Реализуйте версионное хранилище ключей. Каждый `Set` и `Delete` создает монотонную версию; `GetAt(key, version)` должен возвращать значение ключа на указанной версии.

**63.**

Реализуйте склад с остатками, резервами и операциями `Reserve`, `Commit`, `Cancel`. Повтор операции с тем же ID должен быть идемпотентным, а частичный резерв нескольких товаров запрещен.

**64.**

Реализуйте `TTLCache[K comparable, V any]` с методами `Set`, `Get`, `Delete`. Просроченные записи удаляются лениво; источник времени должен передаваться зависимостью для тестов.

**65.**

Реализуйте LRU-кэш фиксированной вместимости с `Get` и `Put` за  $O(1)$ . Индекс элементов храните в карте, порядок использования - в двусвязном списке.

**66.**

Реализуйте LFU-кэш фиксированной вместимости. `Get` и `Put` должны работать за амортизированное  $O(1)$ ; при равной частоте вытесняйте элемент, который использовался раньше.

**67.**

Реализуйте fixed-window rate limiter по пользователям. Для каждого ключа разрешается не более `N` запросов за окно; смена окна не должна требовать полного обхода карты.

**68.**

Реализуйте sliding-window rate limiter по пользователям с точностью до отдельного запроса. Старые отметки времени очищайте при обращении к конкретному пользователю.

**69.**

Дан поток `Event{ID string; At time.Time}` в неубывающем порядке времени. Пропускайте повтор ID, если предыдущее событие с этим ID было не более чем `window` назад; память должна очищаться от устаревших ID.

**70.**

Реализуйте хранилище сессий с абсолютным сроком жизни и продлеваемым сроком бездействия. Метод `Get` должен различать отсутствующую, истекшую и активную сессию.

**71.**

Постройте инвертированный индекс документов `map[string]map[int][]int`, где для каждого нормализованного слова хранятся ID документов и позиции слова. Добавьте удаление документа из всех частей индекса.

**72.**

Над инвертированным индексом реализуйте запросы `AND`, `OR` и `NOT`. Результат должен содержать отсортированные уникальные ID документов.

**73.**

Реализуйте поиск фразы из нескольких слов по позициям в инвертированном индексе. Фраза должна находиться только при последовательных позициях слов в одном документе.

**74.**

Дан поток логов `Log{Service string; At time.Time; Level string}`. Постройте агрегаты количества по сервису, уровню и пятиминутному бакету; выдача должна быть хронологической и детерминированной.

**75.**

Реализуйте разреженный временной ряд `map[int64]float64` с агрегацией по бакетам: `sum`, `count`, `min`, `max`, `avg`. Границы бакетов должны корректно работать для времени до `Unix epoch`.

**76.**

Реализуйте `consistent hash ring` с виртуальными узлами. Нужны операции добавления и удаления физического узла и поиск владельца ключа; коллизии хешей должны обрабатываться явно.

**77.**

Реализуйте потокобезопасную шардированную карту `ShardedMap[K comparable, V any]`. Предусмотрите `Load`, `Store`, `Delete`, `LoadOrStore` и согласованный снимок всех элементов.

**78.**

Реализуйте потокобезопасную карту счетчиков. Метод `Add(key, delta)` должен возвращать новое значение, а `SnapshotAndReset` - атомарно отдавать текущий снимок и обнулять все счетчики.

**79.**

Реализуйте параллельный подсчет частоты слов в наборе больших строк. Число воркеров задается параметром; объединение локальных карт должно исключать гонки и утечки горутин при отмене контекста.

**80.**

Реализуйте аналог `singleflight` для ключей-строк: одновременные вызовы `Do(key, fn)` должны разделять один запуск `fn` и получать одинаковый результат. После завершения ключ удаляется из карты.

**81.**

Реализуйте конкурентный memoizer для `func(context.Context, string) (Value, error)`. Одновременные запросы одного ключа объединяются, успешный результат кэшируется, ошибка не кэшируется, отмена одного ожидающего не отменяет остальных.

**82.**

Дан набор пакетов и зависимостей с ограничениями версий. Напишите резолвер, выбирающий по одной версии каждого пакета или возвращающий конфликт. Циклические зависимости и откат неудачного выбора обязательны к обработке.

**83.**

Напишите `FlattenJSON(v map[string]any) (map[string]any, error)` для вложенных объектов и массивов. Путь кодируйте так, чтобы ключи с точками, скобками и обратными слешами восстанавливались без коллизий.

**84.**

Напишите обратную функцию `UnflattenJSON(flat map[string]any) (map[string]any, error)`. Она должна выявлять конфликтующие пути, неверные индексы массивов и попытки одновременно считать узел значением и контейнером.

**85.**

Реализуйте `DeepMerge(dst, src map[string]any, policy Policy)`. Политика задает правила для объектов, массивов, `nil` и несовпадающих типов; при ошибке `dst` должен остаться неизменным.

**86.**

Реализуйте сравнение двух графов значений `map[string]any`, допускающих циклические ссылки через мапы и слайсы. Нельзя использовать `reflect.DeepEqual`; функция не должна заикливиться.

**87.**

Напишите детерминированную канонизацию `map[string]any` в байты для последующего хеширования. Одинаковые данные должны давать одинаковый результат независимо от порядка мап; числа разных типов нельзя случайно отождествлять.

**88.**

Определите формат операций и реализуйте `DiffJSON(a, b map[string]any) []Op` и `ApplyJSON(base map[string]any, ops []Op)`. Применение корректного diff к `a` должно давать структуру, равную `b`; неверная операция не должна частично менять `base`.

**89.**

Реализуйте маршрутизатор HTTP-путей на trie с мапами дочерних сегментов. Поддержите статические сегменты, параметры `{id}` и завершающий wildcard `*path`; конфликтующие маршруты должны отклоняться.

**90.**

Реализуйте разрешение переменных вида `${a.b.c}` в строковых шаблонах над `map[string]any`. Поддержите значения по умолчанию, экранирование и обнаружение циклов, когда значения сами содержат ссылки.



**91.**

Реализуйте потокобезопасный order book. Заявки индексируются по ID, объем агрегируется по ценовым уровням в мапах; Add, Cancel и Replace должны сохранять согласованность, а лучшие уровни выдаются в правильном порядке.

**92.**

Реализуйте потокобезопасный pub/sub-брокер в памяти. Подписки хранятся как множества по темам; отмена подписки идемпотентна, публикация не держит глобальную блокировку во время отправки и не паникует при параллельной отмене.

**93.**

Реализуйте DNS-подобный кэш с положительным и отрицательным TTL. Одновременные промахи одного имени объединяются, устаревший успешный ответ можно вернуть в пределах stale-интервала при ошибке обновления.

**94.**

Реализуйте менеджер блокировок по произвольным строковым ключам. Захват нескольких ключей одной операцией не должен приводить к дедлоку; неиспользуемые записи блокировок должны удаляться из внутренней карты.

**95.**

Реализуйте планировщик задач с уникальным ID, временем запуска и состояниями. Повторное добавление ID запрещено; Cancel и извлечение готовых задач должны атомарно согласовывать карту и очередь приоритетов.

**96.**

Реализуйте write-ahead log для in-memory `map[string][]byte`. После каждого подтвержденного Set или Delete восстановление из WAL обязано давать то же состояние; оборванная последняя запись не должна портить предыдущие.

**97.**

Реализуйте in-memory `map[string][]byte` с WAL и снимками. Создание snapshot и ротация WAL должны быть атомарными; восстановление после сбоя в любой точке ротации не должно терять подтвержденные операции.

**98.**

Реализуйте MVCC-хранилище ключей с версиями, снимками чтения и оптимистической фиксацией транзакций. Конфликт записи после версии снимка должен отклонять весь commit без частичных изменений.

**99.**

Реализуйте шардированный LRU-кэш с общим лимитом памяти в байтах. Параллельные операции должны быть безопасны, вытеснение - соблюдать общий лимит, а вычисление размера и удаление не должны вызывать пользовательский код под глобальной блокировкой.

**100.**

Реализуйте мини-социальный граф на мапах: добавление и удаление пользователей, взаимная дружба, блокировки, общие друзья и рекомендации второго круга. Все составные изменения должны быть атомарны; рекомендации сортируются по числу общих друзей, затем по ID.